Laboratory Instructions.

Control Engineering II

Temperature Control

Study program: Bachelor Mechatronics, Design & Innovation

Responsible lecturer: Lukas Wild

Date: Winter Term 2026

Contents

1 Introduction **1**

1.1 Hardware 3

1.2 Software 4

2 Preparatory Work for lab session 1 **9**

3 Tasks for lab session 1 **9**

3.1 Open loop step response 9

3.2 Plant identification and Open Loop simulation 10

4 Preparatory Work for lab session 2 **11**

5 Tasks for lab session 2 **12**

5.1 Controller Design Variant A 12

5.2 Controller Design Variant B 12

5.3 Measurements on real system and tuning 12

5.4 Closed loop simulation 13

5.5 2DOF control 13

5.6 Optional: Trajectory for a sintering stove 13

A Electrical schematic of amplifier board **II**

B Setup for MATLAB 2025b or older **III**

B.1 Software III

1 introduction

Temperature control plays a crucial role in various industrial processes, ensuring product quality, operational efficiency, and safety. In control engineering, precise temperature regulation is particularly vital in applications such as the sintering of refractory metals, where the formation of strong, durable materials hinges on maintaining optimal thermal conditions. Deviations in temperature can lead to suboptimal microstructural properties, adversely affecting mechanical performance. Thus, the implementation of advanced temperature control systems is essential for enhancing productivity and achieving desired material characteristics in industries ranging from aerospace to electronics. This laboratory aims to explore the principles and techniques of temperature control, providing hands-on experience that underscores its significance in modern manufacturing processes.

The learning targets for this lab are the following:

- Plant identification of a heating system
- Discretization of a plant model
- Digital PI(D) controller design via Ziegler Nichols Method or Direct Discrete Time Design
- Simulation of discrete controlled system in Simulink
- Measurements on real heating system with controller parameters implemented in a digital controller
- 2DOF control using trajectory generation
- Code generation with Simulink on a micro-controller

2DOF control in industries

However in industries not only the control of a single quantity (e.g. temperature) is important but also its curve. For temperature depending processes it plays a significant role for example which slope is applied for heating up or debiniding. This is illustrated by Figure 1 at the example of a sintering process. Therefore a trajectory planning is utilized to give a specific curve for a process.

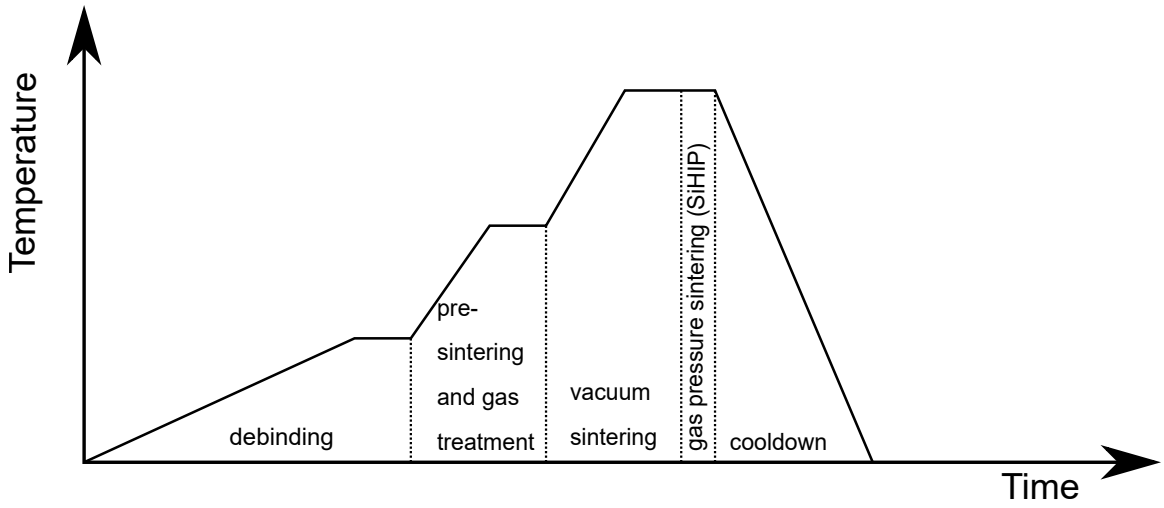


Figure 1: Example of a temperature profile for sintering of refractory metals. The process can take up to 24h and reach temperatures of 1450 °C.

For this reason 2DOF (Two Degrees of Freedom) control is commonly used, which refers to a control system design that allows independent control over two variables. In the context of control theory, this typically involves controlling both the process output (temperature) and the control action (in this example Duty Cycle) simultaneously.

In many applications, 2DOF control is used to achieve better performance by decoupling the control of the output from the feedback mechanism. This can lead to improvements in response time, stability, and disturbance rejection.

For more information the author highly recommends to read [1].

1.1 HARDWARE

The hardware for the laboratory will be provided by the lecturer and is depicted in Figure 2. The system uses a STM32 Nucelo Board as micro-controller, in combination with a $150\,\Omega$ resistor and a temperature sensor LM35. The micro-controller outputs a PWM signal, with varying duty cycle. This is used as input for a MOSFET (FDB8880) that is mounted on a separate board, for the schematic consider Figure 15. The MOSFET is needed as an amplifier, as the output of the micro-controller can not be used to directly drive the heating resistor. The power input needs to be connected to $+12\,\mathrm{V}$, as it is shown.

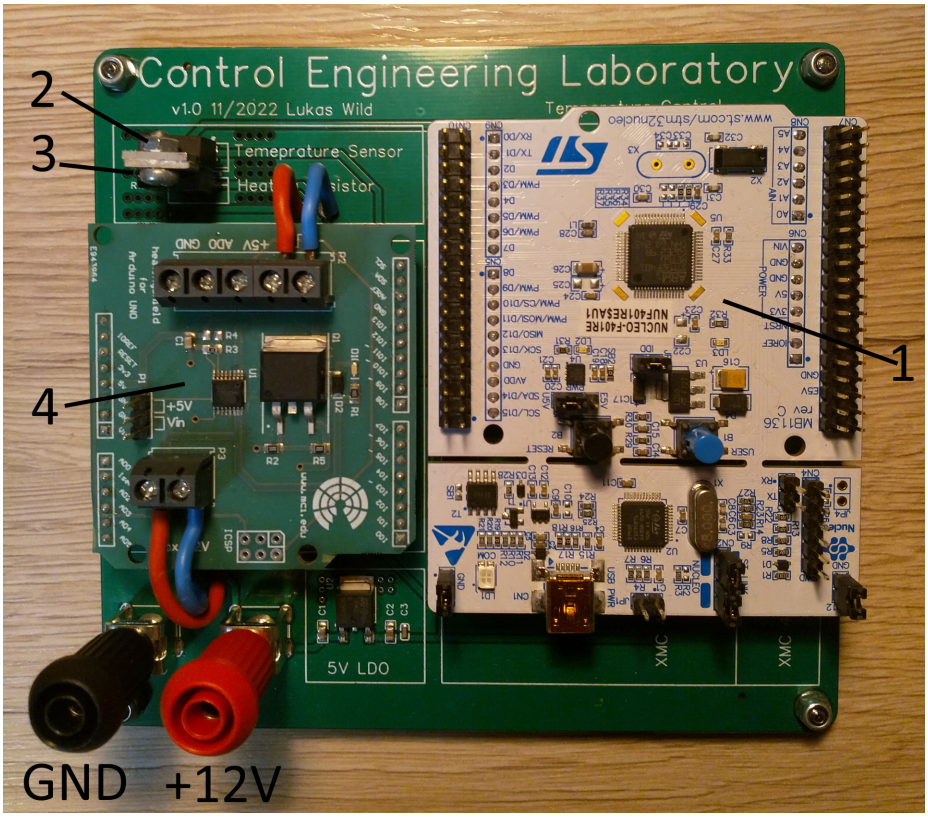


Figure 2: Heating system plant with 1: as micro-controller (STM32 Nucelo Board), 2: LM35 as temperature sensor, 3: heating resistor and 4: amplifier (MCI heatingshield)

This means, that by controlling the Duty Cycle D of the MOSFET the voltage drop V_D and also the power of the resistor is controlled. That allows to control the temperature of the resistor in the end. If the concept of PWM switch control is unknown to the reader, the author recommends to review it, as it is mandatory for the laboratory.

Anyways for all industry processes for a given system an operating point is needed, in order to define a working range for the system. For the lab we will be using an operating point of D=25 %.

1.2 SOFTWARE

This chapter shows the installation, setup, and usage of the required software for the lab. First of all, it is necessary to install **MATLAB 2026a** or higher [1] for the laboratory.

Further more the following MATLAB Add-Ons and standalone tools are required:

- Download and install STM32CubeMX (link) and make sure to download **version 6.16.0**.
- Download and install STM32CubeProgrammer (link) and make sure to download **version 2.22.0**.

  Note: It is not necessary to create a user account to download the STM-Tools, there is an option to download as guest.
- Download and install STM32 Microcontroller Blockset (link) or via Add-On Explorer

STM32 Microcontroller Blockset Configuration

After the installation of the STM32 Microcontroller Blockset you need to configure it. Go to Add-On Explorer and continue as shown in Figure 3.

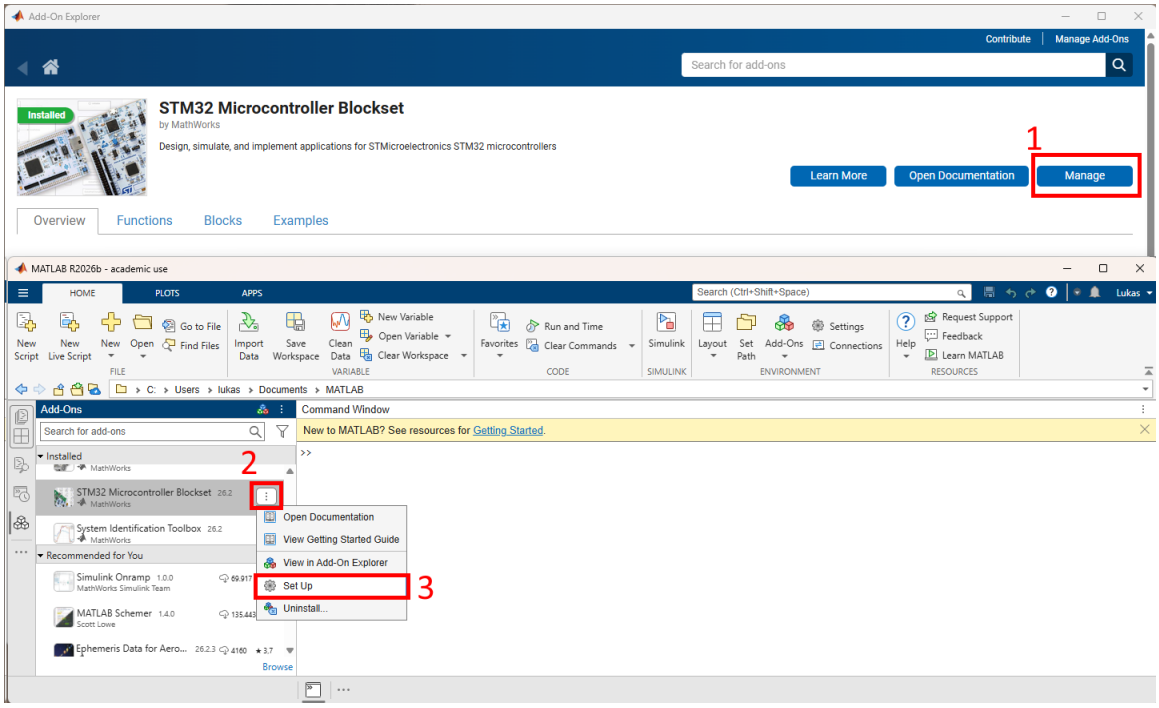
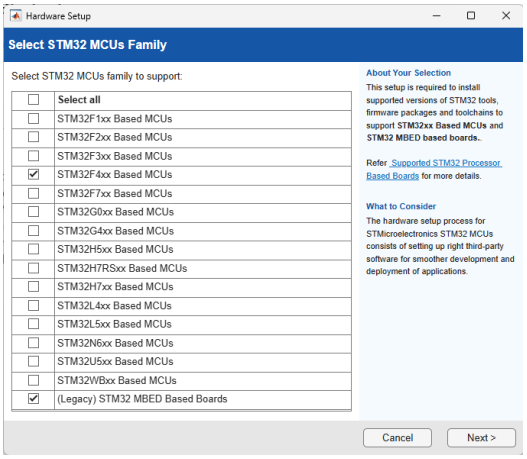


Figure 3: Open Configuration of STM32 Microcontroller Blockset

[1] There has been a change in the way MATLAB handles the STM32 Nucleo Boards support with release 2026a; there is also a guide on how to set it up for version 2025b and older, which can be found in Appendix B

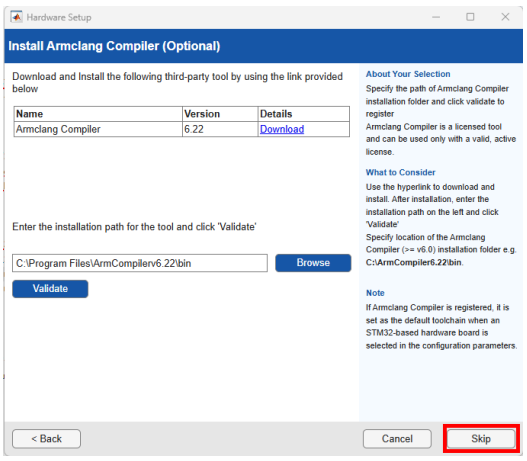
When the configuration window opens, check STM32F4xx Based MCUs **AND** STM32 MBED Based Boards , see Figure 4a and hit next. You are asked to install third-party tools, see Figure 4b, which you have to do and wait until finished, then click next.



(a) STM32 MCU family (b) Install Third Party Tools

Figure 4: STM32 Microcontroller Blockset Configuration 1

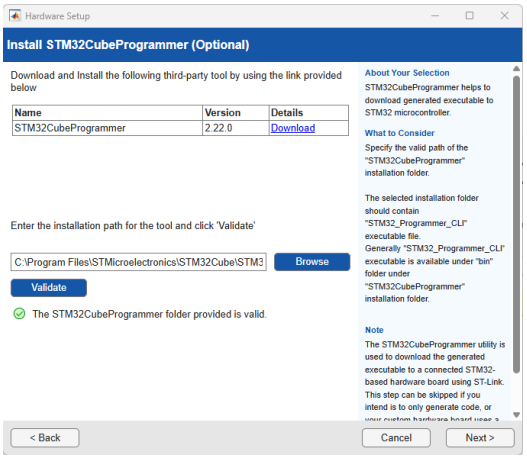
A window opens, asking to install the Armclang Compiler, which is not needed, therefore Skip (Figure 5a). If you have not already installed STM32CubeMX you are now asked to do so. Next, you are asked to validate the path to the installation of the STM32CubeMX software, Figure 5b. Go to browse and find the folder where you installed the software, then press validate. If the green check does not appear, you may have installed the software incorrectly.



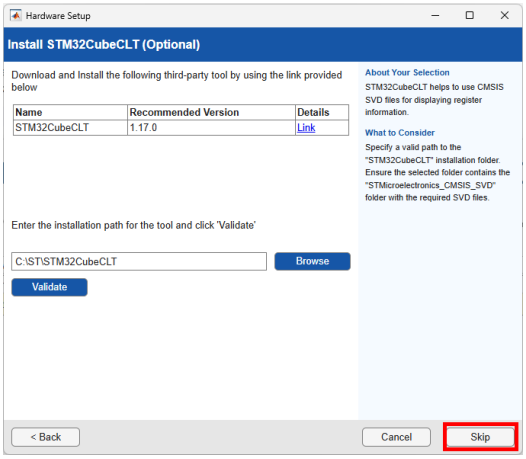
(a) Skip Armclang Compiler (b) validate STM32CubeMX

Figure 5: STM32 Microcontroller Blockset Configuration 2

On the next page the same is done for the STM32CubeProgrammer, Figure 6a. After that you are asked to install STM32CubeCLT, we can skip that, Figure 6b.



(a) validate STM32Cube Programmer

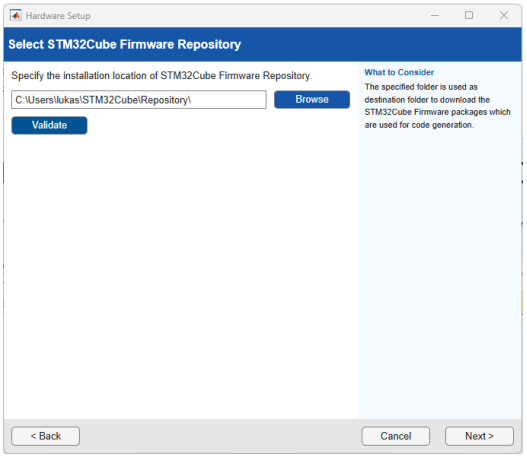


(b) Skip STM32CubeCLT

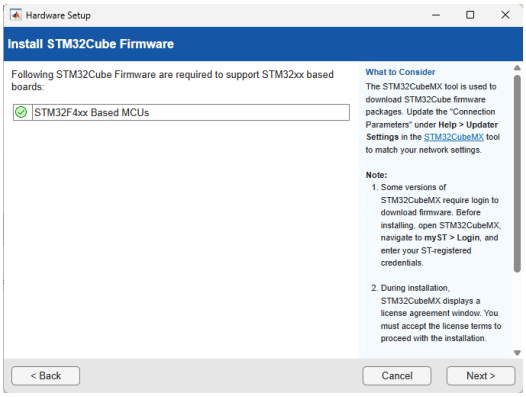
Figure 6: STM32 Microcontroller Blockset Configuration 3

Next you need to provide a folder where the STM32Cube firmware is installed to. This can be any folder, however it may be advisory to use the MATLAB workspace folder that you later want to use in the lab. To check if the folder works press validate, Figure 7a.

After that, the last thing left to do is installing the STM32Cube Firmware, Figure 7b. When this is done the configuration is completed.



(a) Select folder for STM32Cube firmware



(b) Install STM32Cube firmware

Figure 7: STM32 Microcontroller Blockset Configuration 4

Note: When the installation fails the first time with [ERROR] WebUtils:658 - Updater is busy just try it again. It is not a bug, its a feature.

Usage of Hardware Support Package

Download the simulation from Sakai, it contains the Simulink model used to record the step response. In the top bar of Simulink you should see the hardware tab, shown in Figure 8.

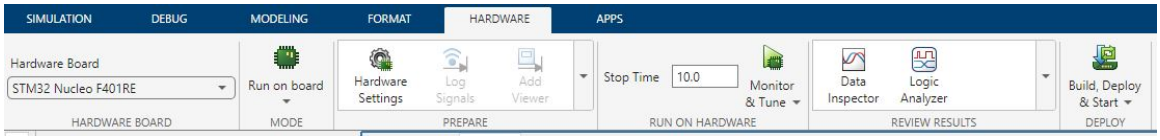


Figure 8: Simulink Top Bar for Hardware Support Package

From there you can select in which way the simulation should be run on the hardware.The different options are:

- Run on board (External Mode): The simulation is running on the real hardware in real time.
- Connected IO (inputs/outputs Mode): The Simulink model is running on the computer, while data is obtained on the STM32 micro-controller. This means while simulating it is possible to communicate with the target.
- Build, Deploy & Start: Code is generated, downloaded to the target hardware and the program is started.
- Monitor & Tune: At first start the compiled Simulink program is downloaded, but afterwards only parameters are changed on the hardware. **This is the mode used during the laboratory.**

To start the simulation Monitor & Tune is used, which changes the appearance of the top bar slightly. Now there is a stop button and right next to it is the Data Inspector, Figure 9. With the Data Inspector the signals from the simulation are logged, which means this tool will be used to record the step response.

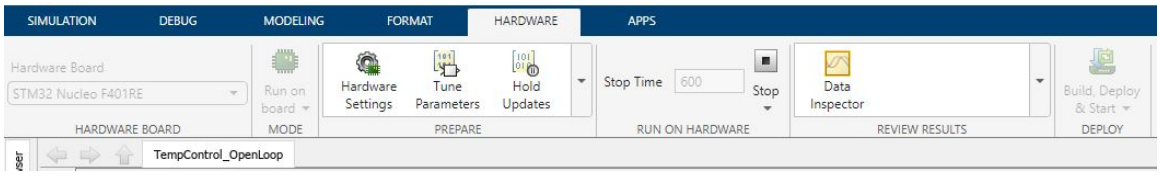


Figure 9: Top Bar when simulation is running

For more information about working with STM32 micro-controllers and Simulink the author recommends watching the following Youtube Playlist:

https://www.youtube.com/playlist?list=PLn8PRpmsu08oqK5APQFLUCnJvw3jOhPR2

Changing the Serial port

Before starting the simulation in the lab, make sure to select the correct serial port. Check your serial port under windows device manager / STMicroelectronics STLink Virtual COM Port. To change the Serial port in the Simulink model open the hardware settings and go to Hardware Implementation / Connected IO as depicted in Figure 10. Then also go to External Mode and enter the same one.

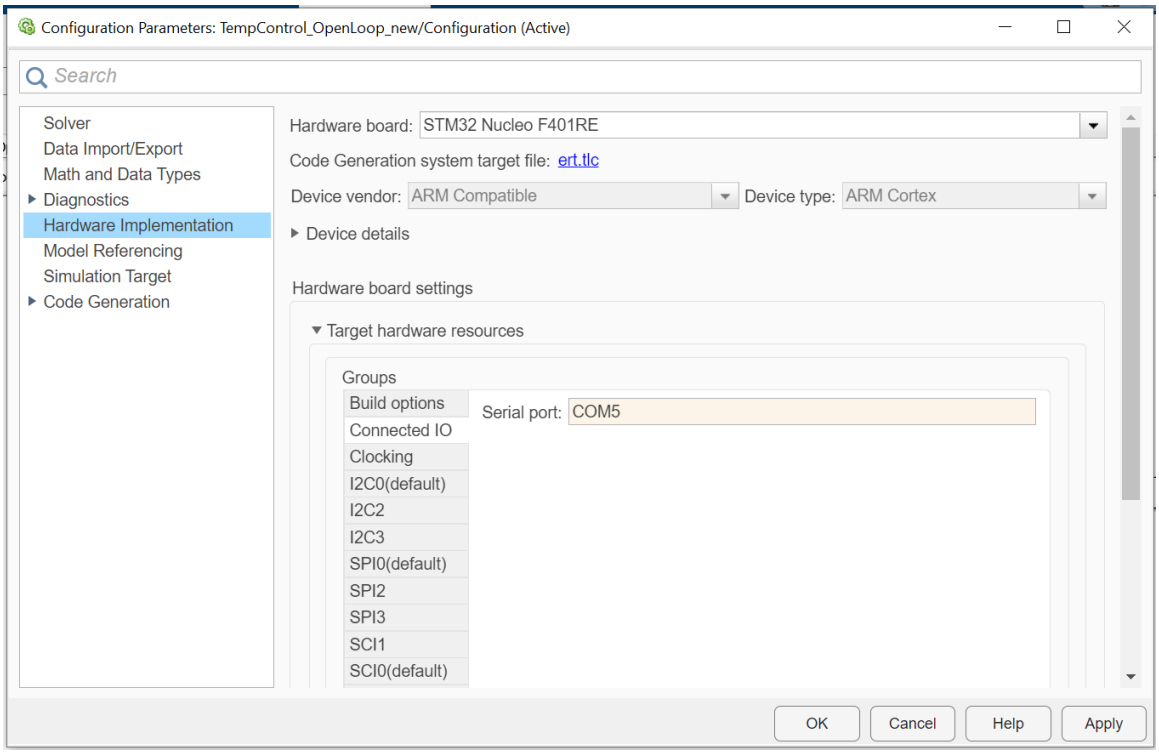


Figure 10: Changing the Serial Port

2 preparatory work for lab session 1

- Install the required software support packages mentioned in chapter 1.2
- Consider Figure 11 and find the gain K_{meas}, that is used to scale the measurement values. Check the datasheet of the LM35 provided on Sakai, the analog reference voltage of the STM32 Nucleo Board is 3.3 V.

  Note: The ADC-Conversion is already considered inside the Analog Read block.
- Create a MATLAB script that can identify a $PT_1(T_t)$ system by using a provided step response (Hint: lsqcurvefit). An example step response is provided by the lecturer on Sakai, this is **NOT** the actual step response.
- Review one of the two controller design methods menitioned below

  Variant A: Ziegler-Nichols method 1 for a PT_1T_t system

  Variant B: Direct Discrete Time design with pole placement

Note: The control system can be sufficiently modeled as a PT_1 system for direct discrete time design method as the dead time T_t is relatively small compared to τ. However for Ziegler-Nichols Method a T_t is required to find the controller parameters, therefore if this method is chosen a PT_1T_t system needs to be identified.

3 tasks for lab session 1

In this section all the tasks to be done in session 1 in the laboratory are listed. Please make sure to include all the results an discussion in your report. Do not exceed 15 pages and keep in mind to follow the MCI laboratory guidelines. Usually it is enough time in the lab to finish all tasks in time, if you have left over time consider chapter 5.

3.1 OPEN LOOP STEP RESPONSE

- Open and start the provided Simulink model, shown in Figure 11.
- The step response is recorded at an operating point of D=25 %, while the step has an amplitude of D=10 %.
- To reach the operating point first heat up until you reach a steady temperature (after several minutes).

- Then use the Start Step switch and log the temperature and the duty cycle. Use the gain K_{meas} that you found during the preparation.
- Open the Real Time Viewer and Log the duty cycle (step input) and the temperature. Make sure you log the temperature until a steady state is reached. The maximum time is set to 10 min, you may expand it.

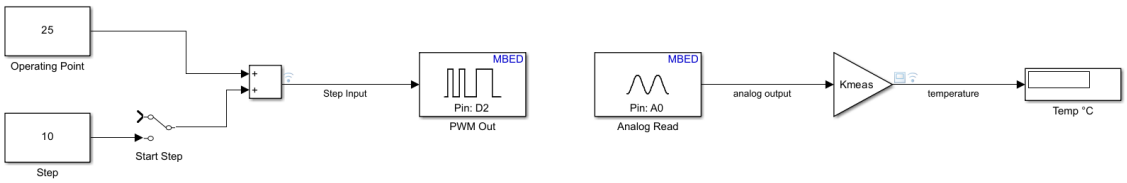


Figure 11: Open Loop system for recording the step response

3.2 PLANT IDENTIFICATION AND OPEN LOOP SIMULATION

- Find the parameters K, τ (and T_t for Variant A) of the system by utilizing your prepared script.

  Note: If you are not able to determine T_t (because of noise, etc), use 0.2 s
- Build an open loop simulation with your fitted system (in s-domain).
- Compare your simulation with the real system and make sure it is similar (otherwise you may check the parameters).

4 preparatory work for lab session 2

- If you have not already, review one of the control design methods (Variant A or B) and prepare it in the script.
- Create a MATLAB function that implements the feed forward control and the trajectory planning according to figure 12 (The trajectory planning and $K_v(s)$ can be inside the same MATLAB function.

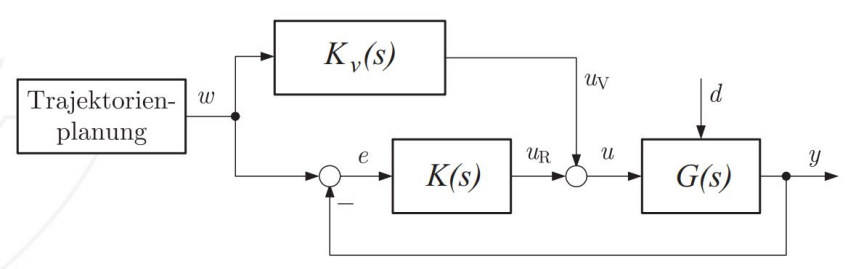


Figure 12: Feedforward Control with Trajectory Planning [2].

- Use the trajectory given by figure 13 and implement it in a MATLAB-function. This will be $w(y_{ref})$ used for the trajectory planning.

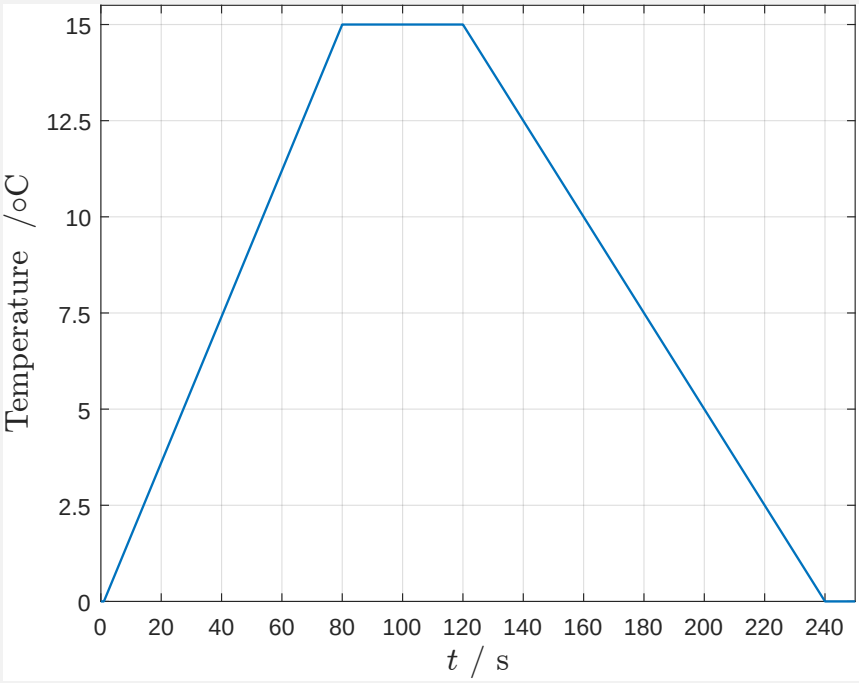


Figure 13: Temperature trajectory relative to ambient temperature

5 tasks for lab session 2

5.1 CONTROLLER DESIGN VARIANT A

- Design a digital PI controller by using the Ziegler-Nichols Method. If you were not able to determine T_t (because of noise, etc), use 0.2 s.

5.2 CONTROLLER DESIGN VARIANT B

- Find a real pole location in z-domain that has a settling time of $t_s = 20$ s.
- Design a digital PI controller (for example with backward Euler) by using direct discrete time design and the found pole location.

5.3 MEASUREMENTS ON REAL SYSTEM AND TUNING

- Rearrange the open loop system from figure 11 and expand it to implement the closed loop controller.
- Do not forget to add a saturation after the PI controller (think of values that make sense).
- Give a temperature input of 35 ° and observe the closed loop step response. Log all the signals in the Data Inspector!
- Check if the controller works and if you can finetune the step response.
  - How do you reduce overshoot?
  - How is steady state error eliminated?
  - What can be done to speed up the system?
- When you feel confident with your controller record a step response from ambient temperature to 35 ° and save it for your report. Measure the overshoot (P.O.) and the settling time of the system.
- Optional: Implement a first order low-pass filter to smooth the measurement signal. In order to filter out the switching frequency, the cutoff frequency f_{LP} for the filter should be 100 times smaller than the switching frequency: $f_{LP} = 0.01 \cdot f_s$. Do not change the gain, that means $K_{LP} = 1$.
- Optional: Discuss why it makes sense / no sense to use a PID controller. What is the derivative part responsible for?

5.4 CLOSED LOOP SIMULATION

- Expand your simulation of the real system with the PI controller and close the loop.
- Again don not forget to implement a saturation after the PI controller.
- Also record a step response to 35 ° and compare it with your measurements. What are differences and why?

5.5 2DOF CONTROL

- Modify the system to implement the prepared 2DOF control with the trajectory from figure 13.
- Take a measurement on the real system and compare to the step response from before. Measure the overshoot (P.O.) and the settling time of the system.
- Also expand you simulation to use the 2DOF control concept and compare to the real measurement.
- Optional: Disconnect the feed forward part (u_{ref}) and check how the system responds with only the trajectory y_{ref}.

5.6 OPTIONAL: TRAJECTORY FOR A SINTERING STOVE

- If you managed to come until here congratulations! However if you still have time you can try to implement the trajectory from figure 14, which is a simplified version of figure 1.

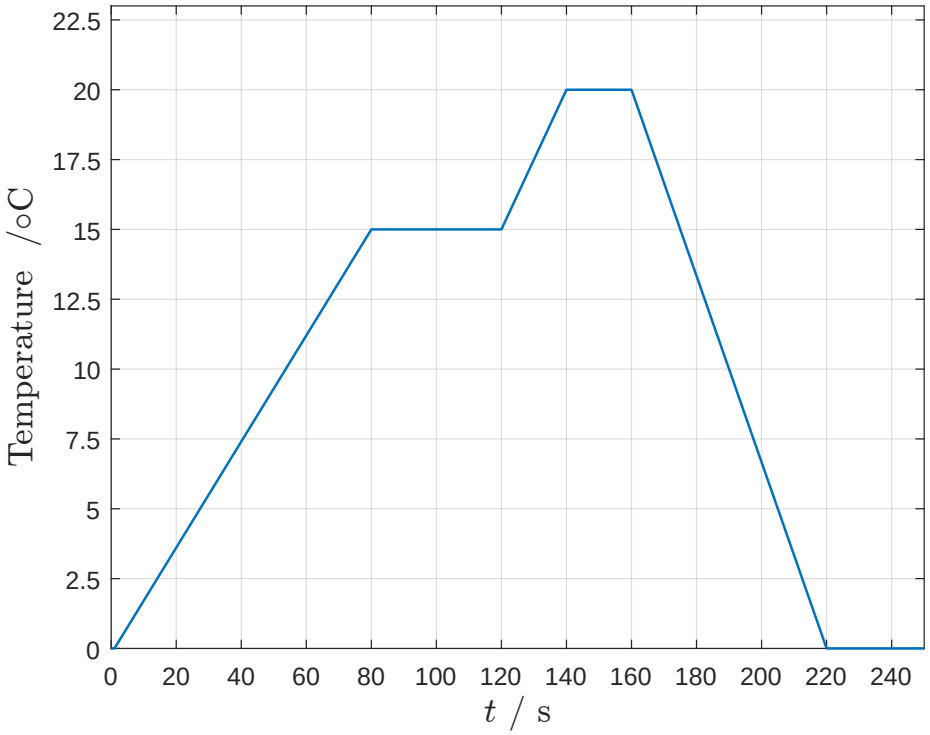


Figure 14: Optional Temperature trajectory for a simplified sintering process

- Take a measurement on the real system and compare to the step response from before.
- Also expand your simulation to use the 2DOF control concept and compare to the real measurement.

A electrical schematic of amplifier board

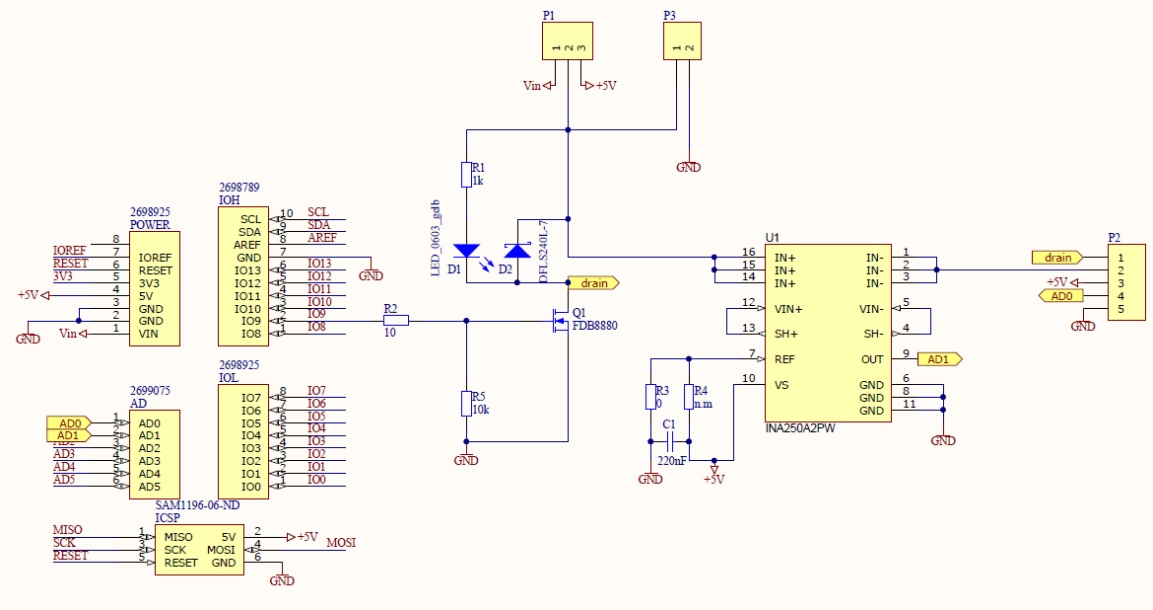


Figure 15: Amplfier board schematic

B setup for matlab 2025b or older

B.1 SOFTWARE

This chapter shows the installation, setup, and usage of the required software for the lab for MATLAB version 2025b and older.

To start the following MATLAB Add-Ons and standalone tools are required:

- Download and install STM32CubeMX (link) and make sure to download **version 6.12.0**.
- Download and install STM32CubeProgrammer (link) and make sure to download **version 2.17.0**.
- Download and install Embedded Coder Support Package for STMicroelectronics STM32 Processors (link)
- Download and install Simulink Coder Support Package for STMicroelectronics Nucleo Boards (link)

Embedded Coder Support Package

After the installation of the Embedded Coder Support Package for STMicroelectronics STM32 Processors you are asked to configure it. If this window was closed, you can open it again as shown in Figure 16.

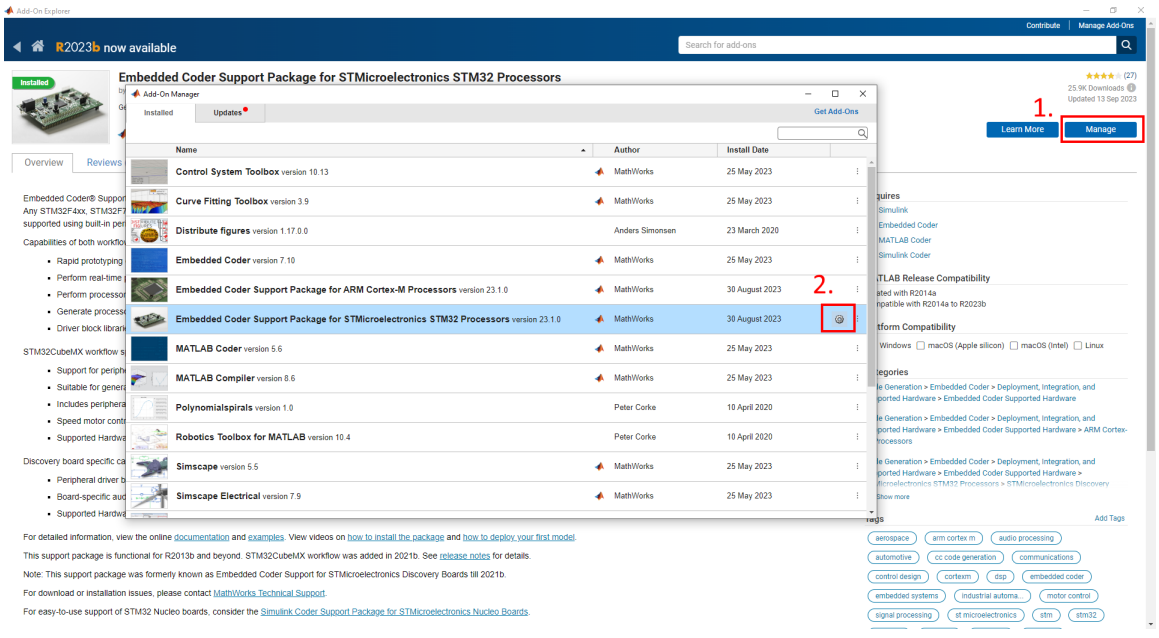
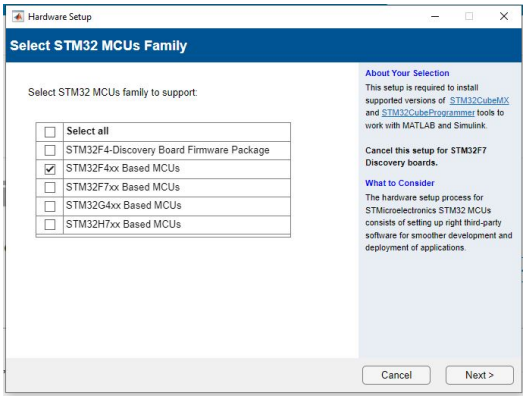
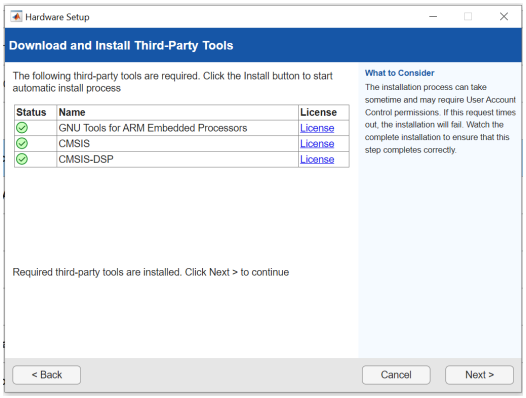


Figure 16: Open Configuration of Embedded Coder Support Package for STMicroelectronics STM32 Processors

When the configuration window opens, check STM32Fxx Based MCUs, see Figure 17a and hit next. You are prompted to download third-party tools, see Figure 17b which you have to download and click next.



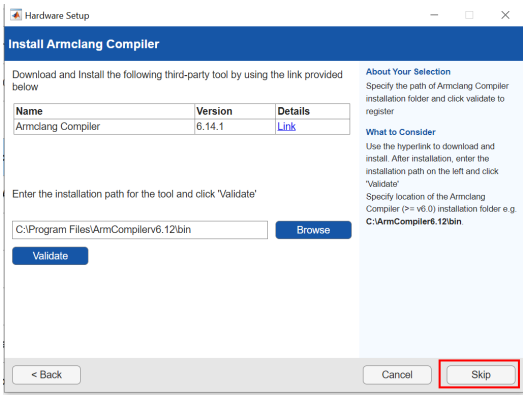
(a) STM32 MCU family



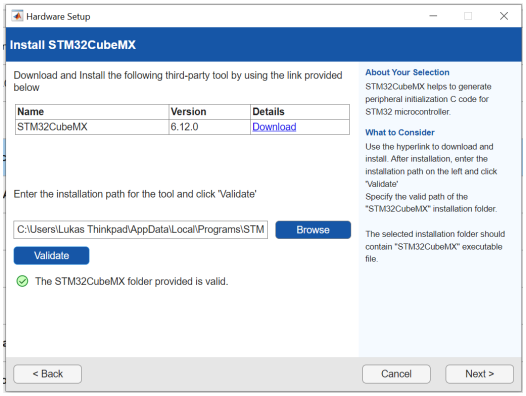
(b) Download Third Party Tools

Figure 17: Support Package Configuration 1

A window opens, asking to install the Armclang Compiler, which is not needed, therefore Skip (Figure 18a). If you have not already installed STM32CubeMX you are now asked to do so. Next, you are asked to validate the path to the installation of the STM32CubeMX software, Figure 18b. Go to browse and find the folder where you installed the software, then press validate. If the green check does not appear, you may have installed the software incorrectly.



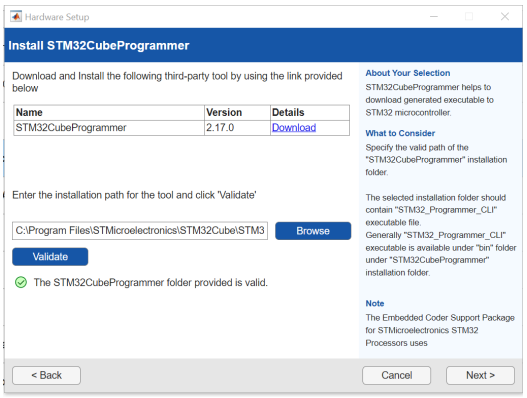
(a) Skip Armclang Compiler



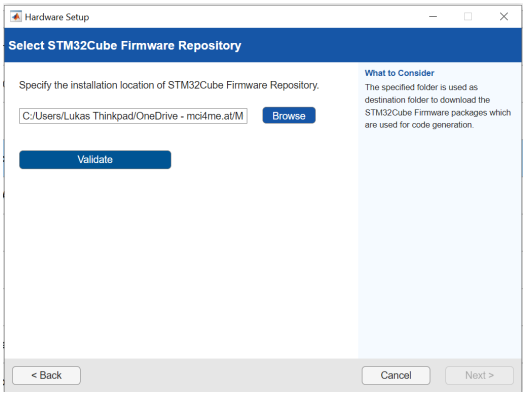
(b) validate STM32CubeMX

Figure 18: Support Package Configuration 2

On the next page the same is done for the STM32CubeProgrammer, Figure 19a. After that you need to provide a folder where the STM32Cube firmware is installed to. This can be any folder, however it may be advisory to use the MATLAB workspace folder that you later want to use in the lab. To check if the folder works press validate, Figure 19b.



(a) validate STM32Cube Programmer



(b) Select folder for STM32Cube firmware

Figure 19: Support Package Configuration 3

After that, the last thing left to do is to install the STM32Cube Firmware, Figure 20. When this is done the configuration is completed.

check if the folder works press validate, Figure 19b.

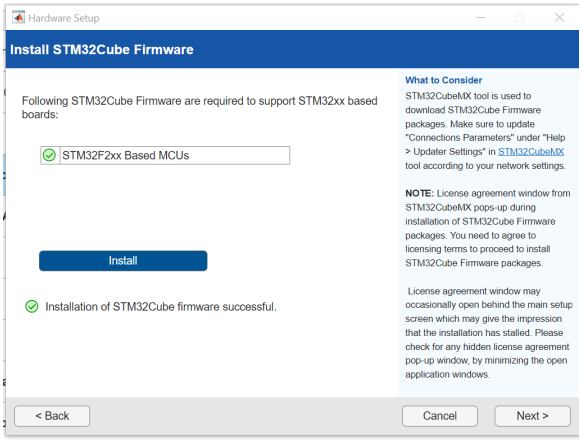


Figure 20: Install STM32Cube firmware

References

[1] P. Kronthaler, **Control Engineering II**. Management Center Innsbruck, Innsbruck, Tirol, 2024.

[2] J. Lunze, **Regelungstechnik Band 1**. Springer-Verlag, 2010.

List of Figures

1 Example of a temperature profile for sintering of refactory metals. 2
2 Heating system plant 3
3 Open Configuration of STM32 Microcontroller Blockset 4
4 STM32 Microcontroller Blockset Configuration 1 5
5 STM32 Microcontroller Blockset Configuration 2 5
6 STM32 Microcontroller Blockset Configuration 3 6
7 STM32 Microcontroller Blockset Configuration 4 6
8 Simulink Top Bar for Hardware Support Package 7
9 Top Bar when simulation is running 7
10 Changing the Serial Port 8
11 Open Loop system for Recording the step response 10
12 Feedforward Control with Trajectory Planning 11
13 Temperature trajectory 11
14 Temperature trajectory relative to ambient temperature 14
15 Amplfier board schematic II
16 Open Configuration of Embedded Coder Support Package for STMicroelectronics STM32 Processors III
17 Support Package Configuration 1 IV
18 Support Package Configuration 2 IV
19 Support Package Configuration 3 V
20 Install STM32Cube firmware V